Step 1 - Understand the Problem and Establish Design Scope

Designing a platform like YouTube is extremely broad because modern video platforms support:

- Uploading videos
- Streaming videos
- Likes/comments
- Recommendations
- Subscriptions
- Live streaming
- Ads
- Analytics
- Playlists
- Notifications

and many other features.

Since a system design interview is usually:

```None
45–60 minutes
```

it is impossible to design the entire ecosystem in detail.

Therefore, the first step is:

```None
Clarify requirements and narrow the scope
```

through interviewer discussion.

Candidate–Interviewer Discussion

Candidate:

What features are important?

Interviewer:

Ability to:

- Upload videos
- Watch videos

Candidate:

What clients do we need to support?

Interviewer:

- Mobile apps
- Web browsers
- Smart TVs

Candidate:

How many daily active users (DAU) do we have?

Interviewer:

None

```
5 Million DAU
```

Candidate:

What is the average daily usage time?

Interviewer:

None

```
30 minutes per user
```

This indicates:

- Very high bandwidth consumption
- Long streaming sessions
- Large CDN traffic

Candidate:

Do we support international users?

Interviewer:

Yes.

This means:

- Global CDN distribution
- Multi-region deployment
- Geo-replication
- International latency optimization

are required.

Candidate:

What video resolutions/formats are supported?

Interviewer:

Most video resolutions and formats.

This implies:

```None
Video transcoding is mandatory
```

because uploaded videos must be converted into:

- Multiple resolutions
- Multiple codecs
- Streaming-compatible formats

Candidate:

Is encryption required?

Interviewer:

Yes.

This means:

- HTTPS/TLS required
- Secure video delivery
- Possibly DRM/protected streaming

Candidate:

Any file size limitations?

Interviewer:

Maximum video size:

```None
1 GB
```

The system focuses on:

- Small videos
- Medium-sized videos

not extremely large movie-scale uploads.

Candidate:

Can we use cloud services from AWS, Google Cloud, or Azure?

Interviewer:

Yes.

This is important because:

```None
Building everything from scratch is unrealistic
```

Modern systems heavily rely on:

- Cloud storage
- CDN services
- Managed databases
- Streaming infrastructure

Final Scope of the System

The interview now focuses on designing a:

```None
Scalable Video Streaming Platform
```

with the following core functionalities.

Functional Requirements

1. Fast Video Upload

Creators should:

- Upload videos reliably
- Resume interrupted uploads
- Upload large files efficiently

2. Smooth Video Streaming

Users should experience:

- Low buffering
- Fast startup time
- Adaptive playback

3. Adaptive Video Quality

The player should dynamically switch between:

- 360p
- 480p
- 720p
- 1080p
- 4K

based on:

- Internet speed
- Device capability
- Network conditions

This is known as:

```None
Adaptive Bitrate Streaming
```

4. Low Infrastructure Cost

Video platforms are expensive because:

- Storage costs are massive
- Bandwidth usage is enormous
- CDN traffic is costly

The system must optimize:

- Storage usage
- CDN caching
- Transcoding efficiency

5. High Availability

The platform should remain operational:

- During server failures
- During network failures
- During peak traffic

Target:

```None
Minimal downtime
```

6. Scalability

The system must scale for:

- Millions of users
- Millions of uploads
- Massive concurrent streams

7. Reliability

The system must ensure:

- No video loss
- Reliable uploads
- Durable storage

Supported Clients

Client Type	Requirements
Mobile Apps	Adaptive streaming, low bandwidth
Web Browsers	HTML5 streaming support
Smart TVs	Large-screen optimized playback

Key Challenges Identified

Challenge 1: Huge Video Files

Even with:

```None
1GB upload limit
```

storage requirements grow rapidly.

Challenge 2: Global Streaming

International users require:

- Regional CDN servers
- Multi-region deployment
- Low latency delivery

Challenge 3: Video Processing

Uploaded videos need:

- Transcoding
- Thumbnail generation
- Compression
- Encryption

These are:

```None
CPU-intensive asynchronous tasks
```

Challenge 4: Bandwidth Optimization

Video streaming dominates internet traffic.

Optimizations needed:

- CDN caching
- Chunked streaming
- Adaptive bitrate delivery

Challenge 5: Device Compatibility

Different devices support:

- Different codecs
- Different resolutions
- Different bandwidth levels

The platform must generate:

```None
Multiple encoded versions
```

of every video.

Estimated System Characteristics

Given:

- 5 million DAU
- 30 minutes watch time/day

The system likely handles:

- Massive concurrent streams
- Petabytes of storage
- Extremely high outbound bandwidth

Important Technologies Likely Needed

Problem	Technology
Video storage	Object storage (S3/HDFS)
Streaming	HLS/DASH
CDN delivery	CloudFront/Akamai
Metadata storage	SQL/NoSQL DB

Queues	Kafka/RabbitMQ
Transcoding	FFmpeg-based workers
Caching	Redis/Memcached

Core Design Goals

The final design should prioritize:

Goal	Importance
Upload speed	High
Streaming quality	High
Low latency	High
Reliability	High
Scalability	High
Cost optimization	High

Final Understanding

At the end of Step 1, we now clearly understand:

- What to build
- What to exclude
- System scale
- Functional requirements
- Technical constraints

This clarity is essential before proposing:

```
None
High-Level Architecture
```

in the next step.